

Tour Ceylon

How an itinerary is generated — from sign-in to the day-by-day plan

This document follows one traveller from the public home page through sign-in, attraction picks, budget, hotels, and the rule engine that stamps clock times. The itinerary is **not** written by ChatGPT. It is built from what the user already saved in the database.

The journey at a glance

Planning is a six-step wizard. Sign-in sits in front of it. Generation happens on step 4 when the Itinerary page opens.

Step	Screen	What the user does	What is saved
0	Home / Login	Opens Travel, signs in or registers	JWT in the browser; user row in the database
1	Attractions	Picks 1–3 moods, trip length, and places	user_trip_input (trip_id, days, moods, attraction IDs)
2	Budget	Sets total spend and optional custom split	budget_split (food / stay / shopping / transport)
3	Stay	Chooses one hotel per destination city	Hotels written onto the same trip
4	Itinerary	Page opens (or user clicks Generate)	generated_itinerary JSON with days and times
5–6	Discover / Export	Guides, agencies, download PDF	Same itinerary; optional saved references

Rule: unique locations on the trip cannot exceed the number of days. Generation is refused until attractions, budget split, and one hotel per city exist.

1. Home page (before login)

The Travel app home is public. Anyone can browse featured attractions. The Itinerary card and “Get in touch” send a signed-in user to **/attractions**. If they are not signed in, they go to **/login**.

- Home also links Wellness, SOS, and Live Data (other Tour Ceylon modules).
- Featured photos load from **GET /api/attractions** (no login required).
- Planning screens after login sit behind a protected route.

2. Sign in

The login card title is “Sign in to plan your trip”. After a successful sign-in the browser is sent to **/attractions** and the wizard starts.

Ways to get an account

- 1 **Email + password Sign In** — **POST /api/auth/login**. Looks up the user, checks the password, returns a JWT.
- 2 **Register** — **POST /api/auth/register**. Creates a new user (password at least 6 characters) and returns a JWT immediately.
- 3 **Continue with Google** — Google Identity Services on the page; the ID token is sent to **POST /api/auth/google**. A travel user is created or linked by email / Google ID.
- 4 **SOS fallback** — if travel login fails, the app tries Tourist SOS login, then **POST /api/auth/link-account** so one email can work in both apps.

What is stored in the browser

- **access_token** — JWT with identity = user id and role **travel_user**.
- **user** — JSON profile (email, name, picture).
- Every later API call sends **Authorization: Bearer ...**.
- On load, the app checks expiry and calls **GET /api/auth/me**. A stale or invalid token is cleared and the user is sent back to login.

Google-only accounts cannot use a password until they set one via link-account. Police officers use a different username login; that path is not used for itinerary planning.

3. After login: the planning shell

Routes under Attractions, Budget, Stay, Itinerary, Discover, Export, History, and Profile require a valid token. Without one, the user is redirected to login. The sidebar lists the six wizard steps. Progress in the top bar is about 25% → 45% → 65% → 80% → 90% → 100% as they move forward.

- Trip state (trip_id, moods, days, budget, hotels, itinerary) lives in browser memory (**TripContext**) for this session.
- The durable copy is in PostgreSQL, keyed by the logged-in user.
- The top-bar search jumps back to Attractions and filters the catalogue.

4. Attractions — create the trip

This is the first planning screen. The catalogue loads from **GET /api/attractions**. Live Data risk badges may overlay from **/api/risk** (optional; generation does not depend on risk).

What the user chooses

- 1 **Trip length** — number of days (default 7).
- 2 **Moods** — 1 to 3 pills: Adventure, Authentic, Curious, Excited, Explore, Happy, Healing, Peaceful, Relaxed, Spiritual. These later drive Auto start time.
- 3 **Category filter** — Wild, Scenic, Pristine, Heritage, Essence, Thrills (or All).
- 4 **Places** — at least one attraction. Search matches name, city, or details.

Hard rules before Confirm

- At least one mood and one attraction.
- Number of **distinct destination cities** must be between 1 and the number of days (a 7-day trip can visit at most 7 cities; several places in the same city is fine).

Confirm calls **POST /api/trip-input** with days, selected_moods, and finalized_attractions (IDs in pick order). The server:

- 1 Checks the user from the JWT and validates IDs exist.
- 2 Merges moods from the chosen places with the pills (max 3 stored).
- 3 Creates a **user_trip_input** row (status in_progress).
- 4 Builds a day/city skeleton (**get_day_plan()**) and returns trip_id.

The browser stores trip_id and goes to Budget. Without this save, later pages cannot allocate money or list hotels.

5. Budget — split the spend

The user sets a total budget (default 2500, treated as USD in the UI). Default split is 25% food, 35% stay, 15% shopping, 25% transport. Turning on Manual Adjustment lets them move sliders; they must still sum to 100%. Turning it off resets to those defaults.

Confirm calls **POST /api/budget/split** with trip_id, budget, and optional custom percentages. The server writes **budget_split** amounts and percentages, and stores the total on the trip. Then the browser goes to Stay.

Accommodation search uses the stay slice of this split (per night, converted with LKR_TO_USD_RATE, default 300) to hide hotels that are over budget.

6. Stay — one hotel per city

The page loads **GET /api/accommodation?trip_id=...&room_type=....** Room type is single, double (default), or family. Hotels are grouped by the destination cities already implied by the selected attractions.

- Budget split must already exist; otherwise the API refuses the list.
- The user must pick **one hotel for every destination**.
- If a city has no hotel in budget, they must raise the stay budget or change places.

Continue calls **PATCH /api/trip-input/{trip_id}** with `accommodations_by_destination`. The trip now has attractions, budget, and hotels. The browser goes to Itinerary.

7. Itinerary page — when generation runs

This is the moment the day-by-day schedule is created. The page does not wait for a special “AI” product. It calls the generate API with the trip already saved.

- 1 If hotels are missing, the page asks the user to go back to Stay.
- 2 If a plan already exists for this trip, it loads **GET /api/itinerary/{trip_id}**.
- 3 If none exists (404), it automatically calls **POST /api/itinerary/generate** with `trip_id` and optional `day_start`.
- 4 The user can change start time (or Auto by mood) and click Generate again; the old JSON for that trip is deleted and replaced.

The server refuses generate unless: (1) at least one attraction, (2) one hotel per destination, (3) a budget split. All of those were collected in steps 4–6 after sign-in.

8. What the generator does with that saved trip

Main files: **routes/itinerary.py** (API) → **ai_service.py** (times and day JSON) → **trip_plan_service.py** (city/day split and hotels). The JSON is stored in **generated_itinerary** with source “database”.

- 1 Split destination cities across the trip days.
- 2 Assign each selected attraction to a day.
- 3 Choose the day's start hour (user time, or Auto by mood).
- 4 Stamp clock times on each stop, with lunch and evening limits.
- 5 Attach the hotel already chosen for that city, then save JSON.

8.1 Split cities across days

Places are grouped by destination, in the **order the user picked them**. Total days are shared as evenly as possible. The trip never bounces back to a city after it has moved on.

Example: 7 days, Colombo → Kandy → Galle. Base days = $7 \div 3 = 2$, remainder 1 goes to later cities → about 2 days Colombo, 2 Kandy, 3 Galle. Function: **get_day_plan()**.

8.2 Put attractions on those days

- **One day in a city** — all of that city's selected places go on that day.
- **Several days in a city** — one place per day; extras are stacked on the last day of that block.
- **Extra days with no leftover places** — a leisure slot: “Explore {city}” (free time, not a repeat visit).

Function: **attractions_grouped_by_day()**.

8.3 Choose the start time

The Itinerary page can send a clock time, or leave it empty for Auto.

Mood saved on the trip	Start hour
Spiritual	6:30 AM
Adventure	7:30 AM
Explore / Excited	8:00 AM
Curious	8:30 AM
Happy / Authentic	9:00 AM
Healing	9:30 AM
Peaceful	10:00 AM
Relaxed	10:30 AM
No moods (fallback)	8:30 AM

- User chose a time (for example **8:30 AM**) → that hour is used.
- **Auto (by mood)** or empty → average of the mood hours above. Spiritual (6:30) + Relaxed (10:30) → 8:30.
- The **first place of the day** can push the start later, never earlier: Wild / safari / temple earlier; beach / spa later. Start is clamped between 5:00 AM and 2:00 PM.

Category or name hint	Bias
Wild	6:30 AM
Heritage / temple / religious	6:30–7:00 AM
Thrills / hiking	7:00–7:30 AM
Scenic / waterfall	8:00–8:30 AM
Essence / market	9:00 AM

Pristine / beach	9:30 AM
Museum / spa	10:00–10:30 AM

Functions: `_parse_day_start()`, `_mood_start_hour()`, `_category_bias()` in `ai_service.py`.

8.4 Stamp clock times on each stop

Once the start hour is fixed, stops on that day are spaced with simple gaps. This is not driving time from a map.

Stops that day	Gap between stops
1 place	Only the start time
2 places	About 3 hours apart
3 places	About 2.5 hours apart
4 or more	About 2 hours apart

- If the next slot falls in **12:00–1:00 PM**, it jumps to **1:00 PM** (lunch).
- Nothing is planned after about **8:00 PM** (late slots are pulled back to 7:30 PM).
- Example: start 8:00 AM, three places → about 8:00 AM, 10:30 AM, 1:00 PM.

Function: `_build_day_times()` in `ai_service.py`.

8.5 Attach hotel and save

- Each day gets the hotel already chosen for that city (`hotel_for_day()`).
- The builder returns one JSON object: title, summary, route (for example Kandy → Galle), highlights, days (activities + times + stay), and **source: database**.
- That JSON is stored in **generated_itinerary**. The Itinerary page reads it.

9. After the plan exists

- **Discover** — businesses, agencies, and guides near the trip; user can save references.
- **Export** — download a trip PDF from the same JSON (`GET /api/itinerary/{trip_id}/pdf`).
- **History** — lists generated plans for this signed-in user (`GET /api/itinerary/history`); opening one reloads that trip on the Itinerary page.
- **Log out** — clears the JWT; planning routes require sign-in again.

10. API chain (signed-in user)

When	Method	Purpose
Sign in / register / Google	POST <code>/api/auth/login</code> register google	Issue travel JWT
Refresh session	GET <code>/api/auth/me</code>	Confirm token still valid
Catalogue	GET <code>/api/attractions</code>	List places (public)
Confirm attractions	POST <code>/api/trip-input</code>	Create trip for this user
Confirm budget	POST <code>/api/budget/split</code>	Save percentages and amounts
Open Stay	GET <code>/api/accommodation</code>	Hotels in budget per city
Confirm hotels	PATCH <code>/api/trip-input/{id}</code>	Attach one hotel per city
Open Itinerary / Generate	POST <code>/api/itinerary/generate</code>	Build and save day JSON
Reload plan	GET <code>/api/itinerary/{id}</code>	Read saved JSON

11. What this generator does not do

- It does not call OpenAI (OPENAI_API_KEY can stay empty).
- It does not use live maps, road times, or opening hours.
- It does not reorder cities geographically — it keeps pick order from Attractions.
- It does not invent hotels; it only attaches stays the user confirmed after sign-in.
- It does not run until the user is authenticated and the trip has attractions, budget, and hotels.

Tour Ceylon — sign in → pick places and moods → split budget → choose hotels → generate: city blocks → assign places → start hour from user or mood → spaced times + hotel.